mra.py 中频域插补模块的代码与数学说明手册

2026-04-09

摘要

本文面向仓库中的 `mra.py`，对其中**频域插补模块**及其直接相关的数据流、训练流和测试流做系统说明。文档只聚焦频域专家在多专家门控插补框架中的作用，不展开图卷积专家和异常检测器的完整理论，但会在必要处说明它们与频域模块的耦合关系。报告覆盖以下内容：代码入口与训练入口、数据预处理与窗口化、频域模块的数学原理、输入输出张量形状、联合训练中的损失分解与参数更新路径、训练集上学习到的参数如何在测试集上影响补全结果与异常分数，以及当前数据集与默认配置下的具体维度。为了避免空谈，文中的关键结论均直接对应代码实现，并辅以一次单批次自动求导检查来验证梯度流向。

关键词：多采样率，频域插补，FFT，门控融合，联合训练，异常检测

目录

1 文档范围与核心结论 **3**
1.1 分析范围 . . . 3
1.2 先给结论 . . . 3

2 代码入口与调用定位 **3**
2.1 入口总表 . . . 3
2.2 两个训练入口必须区分 . . . 4

3 数据、窗口与多采样率元数据 **5**
3.1 原始数据与掩码 . . . 5
3.2 标准化后的数值含义 . . . 5
3.3 训练集中的三种采样率 . . . 5
3.4 窗口化后张量形状 . . . 6

4 频域模块在整体模型中的位置 **6**
4.1 整体数据流 . . . 6
4.2 为什么它不直接吃输入窗口 . . . 6

5 频域插补模块的数学原理 **7**
5.1 模块结构概述 . . . 7
5.2 实数 FFT 与半谱表示 . . . 7
5.3 实部、虚部与频谱描述向量 . . . 7
5.4 注意力分支：决定每个频点该修多少 . . . 8

5.5 增强分支：学习复数频谱残差 8
5.6 逆变换回时域 8
5.7 输出头：把频域重构再做一次跨变量修正 9
5.8 默认配置下的具体维度 9

6 从代码看张量如何流动 9
6.1 训练时的逐步张量流 9
6.2 门控层为什么会用到频域残差 10
6.3 测试时的张量流有何不同 10

7 训练入口、损失函数与参数更新 10
7.1 独立训练入口：独立频域脚本 10
7.2 联合训练入口：mra.py 11
7.3 五项损失的精确定义 11
7.4 哪些参数会被哪些损失更新 12
7.5 为什么检测器损失会更新频域模块 13
7.6 训练时额外遮挡的意义 13

8 训练集学到的参数，在测试集里如何起作用 13
8.1 先说参数本身 13
8.2 测试阶段的直接作用 14
8.3 测试阶段的间接作用 14
8.4 一个经常被忽略的事实 15

9 默认配置下的参数、张量和评分链条 15
9.1 与频域模块直接相关的默认超参数 15
9.2 最终异常分数是怎样把频域模块卷进去的 15

10 设计思路、优点与局限 15
10.1 设计思路总结 15
10.2 这样设计的优点 16
10.3 从代码出发能看到的局限 16

11 结论 16

1 文档范围与核心结论

1.1 分析范围

本文只分析以下与频域模块直接相关的部分：

- `freq_reconstruction.py` 中 `FrequencyImputer` 与 `FrequencyOnlyReconstructor` 的实现；
- `mra.py` 中频域专家的接入位置、联合训练入口与测试入口；
- 为了说明张量流和损失传播，不可避免地涉及 `PreImpute`、门控层、窗口构造、标准化器和异常分数计算；
- 图学习专家和 Transformer 检测器只讨论它们与频域模块发生交互的部分，不单独展开全部理论。

1.2 先给结论

结合代码阅读与运行时检查，可以先把最关键的五点结论说清楚：

- 频域模块在 `mra.py` 中**不是独立训练**，而是作为 `TwoExpertGatedImputer` 的一个专家，与 GCN 专家、门控网络和 Transformer 检测器一起做**单优化器**、**单反向传播**的联合训练。
- 频域模块真正接收的输入不是原始零填充窗口，而是 `PreImpute.fill` 产生的**双向时域种子** $\boldsymbol{X}_{\text{seed}}$。这一步显著减弱了直接对零掩码做 FFT 带来的频谱泄漏。
- 频域核心分支本质上是**对每个变量单独做实数 FFT 频谱残差校正**：先把实部和虚部拼接，再用一个注意力分支给出每个频点的门控权重，用另一个增强分支生成复数残差，最后经逆 FFT 回到时域。
- 频域模块在联合训练中不只受到 `freq_loss` 监督，还会受到 `fusion_loss` 和 `detector_loss` 的反向影响。尤其是 `detector_loss` 通过 $\boldsymbol{X}_{\text{complete}}$ 这条路径，会反向更新频域专家、门控层乃至 GCN 专家。
- 在测试阶段，训练集学到的频域参数不仅影响补全值 $\boldsymbol{X}_{\text{complete}}$，还直接或间接影响 `disagreement_score`、`gate_entropy`、`shift_score` 以及最终阈值化异常分数。

2 代码入口与调用定位

2.1 入口总表

模块位置	代码定位	作用
频域核心定义	`freq_reconstruction.py:95-141`	定义 `FrequencyImputer` 和 `FrequencyOnlyReconstructor`。
独立训练入口	`freq_reconstruction.py:153-201`	以随机遮挡重构为目标，单独训练频域模块。

模块位置	代码定位	作用
频域专家接入点	mra.py:270-340	在 TwoExpertGatedImputer 中实例化频域专家，并与 GCN 专家做门控融合。
联合模型封装	mra.py:457-509	将插补器与 Transformer 检测器组合为 FusionAnomalyModel。
联合损失定义	mra.py:512-545	定义 fusion/gcn/freq/detector/graph 五项损失及总损失。
联合训练入口	mra.py:548-621	训练时采样 holdout 掩码，执行前向、反向、梯度裁剪和 AdamW 更新。
重构推理入口	mra.py:624-662	生成 train_completed.csv、test_completed.csv 等补全结果。
统计量采集入口	mra.py:665-722	计算检测分数、专家分歧和门控熵。
主程序入口	mra.py:831-1098	组织数据加载、标准化、窗口化、训练、推理、评分和保存输出。
采样率推断	mra.py:139-150	根据训练集缺失率推断每个变量的采样步长与采样率类别。
训练期额外遮挡	mra.py:162-182	在观测位置上随机采样 holdout 掩码，构造自监督目标。
双向时域预插补	mra.py:185-248	先做正向/反向填充，再生成频域模块的输入种子。
窗口构造	utils/methods/windowing.py:79-167	构建训练插补窗口、训练评分窗口和测试评分窗口。
标准化器	graph_gcn_reconstruction.py:167-206	只用观测值估计均值方差，并把缺失位置变为标准化后的零。

2.2 两个训练入口必须区分

这份代码实际上存在两个“训练入口”，但含义不同：

1. **独立训练入口**：freq_reconstruction.py 中的 train_model。它只训练频域重构模块本身，优化目标是被随机遮挡位置的重构误差。

2. **实际联合训练入口**：mra.py 中的 train_model。当用户运行 mra.py 时，频域模块被放入双专家门控插补器中，与图专家和检测器一起训练。

如果问题是“mra.py 里的频域模块怎么训练”，答案必须以第二条为主；第一条更像独立实验脚本或拆分验证脚本。

3 数据、窗口与多采样率元数据

3.1 原始数据与掩码

`load_csv_glob_with_mask` 会读取 CSV 并把 `NaN` 转成缺失掩码 `missing_mask`。当前仓库默认数据的实际尺寸为：

- 训练集原始数据：$\boldsymbol{X}_{\text{raw}}^{\text{train}} \in \mathbb{R}^{4100\times 18}$；
- 测试集原始数据：$\boldsymbol{X}_{\text{raw}}^{\text{test}} \in \mathbb{R}^{4100\times 18}$；
- 训练/测试掩码：$\boldsymbol{M}_s^{\text{train}}, \boldsymbol{M}_s^{\text{test}} \in \{0,1\}^{4100\times 18}$；
- 总体缺失率约为 0.4889。

这里下标 s 表示 *structural missing mask*，即数据本身原生的缺失模式。

3.2 标准化后的数值含义

`ObservedStandardScaler` 只用**观测值**估计每个变量的均值和标准差。对缺失位置，它先用该变量观测均值填充，再标准化。因此：

$$X_{t,n} = \begin{cases} \dfrac{X_{t,n}^{\text{raw}} - \mu_n}{\sigma_n}, & M_{s,t,n} = 0, \\ 0, & M_{s,t,n} = 1. \end{cases} \tag{1}$$

这意味着标准化后的缺失位置数值上等于 0，但模型并不把 0 当成“真实观测为零”，而是始终配套使用掩码来区分“零值”和“缺失值”。

3.3 训练集中的三种采样率

`infer_rate_metadata` 根据训练集每列的缺失率估计采样步长：

$$r_n = 1 - \frac{1}{T_{\text{all}}} \sum_{t=1}^{T_{\text{all}}} M_{s,t,n}, \qquad s_n = \text{round}\left(\frac{1}{\max(r_n, 10^{-6})}\right), \tag{2}$$

其中 r_n 是第 n 个变量的观测率，s_n 是推断出的采样步长。

对当前默认数据，得到三组变量：

变量组	列索引	缺失率	观测率	推断步长 s_n	`rate_id`
高频组	1–6	0.0	1.0	1	0
中频组	7–12	0.6667	0.3333	3	1
低频组	13–18	0.8	0.2	5	2

因此：

$$\boldsymbol{stride} = [1,1,1,1,1,1,3,3,3,3,3,3,5,5,5,5,5,5], \tag{3}$$

$$\boldsymbol{rate_id} = [0,0,0,0,0,0,1,1,1,1,1,1,2,2,2,2,2,2]. \tag{4}$$

3.4 窗口化后张量形状

默认参数 `seq_len=50`、`stride=1`。窗口构造分成两类：

- `build_windows`：用于训练插补与完整序列补全，允许前端填充；
- `build_standard_windows` / `build_prompt_test_windows`：用于异常评分，不做前端填充，并从较晚位置开始截取稳定窗口。

当前数据集上的实际形状为：

$$\boldsymbol{W}_{\text{imp}}^{\text{train}} \in \mathbb{R}^{4100\times 50\times 18}, \tag{5}$$

$$\boldsymbol{W}_{\text{imp}}^{\text{test}} \in \mathbb{R}^{4100\times 50\times 18}, \tag{6}$$

$$\boldsymbol{W}_{\text{eval}}^{\text{train}} \in \mathbb{R}^{4001\times 50\times 18}, \tag{7}$$

$$\boldsymbol{W}_{\text{eval}}^{\text{test}} \in \mathbb{R}^{4000\times 50\times 18}. \tag{8}$$

测试标签是由 `build_prompt_test_windows` 生成的固定分段标签，前 2000 个窗口为正常，后 2000 个窗口为异常。

4 频域模块在整体模型中的位置

4.1 整体数据流

若记一个训练 batch 的真实标准化窗口为

$$\boldsymbol{X}_{\text{true}} \in \mathbb{R}^{B\times T\times N}, \tag{9}$$

其中默认 $T = 50$、$N = 18$，则 `mra.py` 中与频域模块相关的主流程可以写成：

$$\boldsymbol{M}_h = \text{SampleHoldout}(\boldsymbol{M}_s), \tag{10}$$

$$\boldsymbol{M} = \max(\boldsymbol{M}_s, \boldsymbol{M}_h), \tag{11}$$

$$\boldsymbol{X}_{\text{input}} = \boldsymbol{X}_{\text{true}} \odot (1 - \boldsymbol{M}), \tag{12}$$

$$\boldsymbol{X}_{\text{seed}} = \text{PreImpute}(\boldsymbol{X}_{\text{input}}, \boldsymbol{M}), \tag{13}$$

$$\boldsymbol{X}_{\text{gcn}} = f_{\text{gcn}}(\boldsymbol{X}_{\text{seed}}, \boldsymbol{M}), \tag{14}$$

$$\boldsymbol{X}_{\text{freq}} = f_{\text{freq}}(\boldsymbol{X}_{\text{seed}}), \tag{15}$$

$$\boldsymbol{\Pi} = g(\boldsymbol{X}_{\text{seed}}, \boldsymbol{M}, \boldsymbol{X}_{\text{gcn}}, \boldsymbol{X}_{\text{freq}}, \boldsymbol{rate_id}), \tag{16}$$

$$\boldsymbol{X}_{\text{imp}} = \Pi_{:,:,:,0} \odot \boldsymbol{X}_{\text{gcn}} + \Pi_{:,:,:,1} \odot \boldsymbol{X}_{\text{freq}}, \tag{17}$$

$$\boldsymbol{X}_{\text{complete}} = \boldsymbol{M} \odot \boldsymbol{X}_{\text{imp}} + (1 - \boldsymbol{M}) \odot \boldsymbol{X}_{\text{input}}. \tag{18}$$

频域模块的位置很明确：它既输出自己的专家结果 $\boldsymbol{X}_{\text{freq}}$，又通过门控权重 $\boldsymbol{\Pi}$ 参与最终补全 $\boldsymbol{X}_{\text{complete}}$。

4.2 为什么它不直接吃输入窗口

这是设计上很重要的一点。若直接对 $\boldsymbol{X}_{\text{input}}$ 做 FFT，原始缺失位的零值会在频域中表现成与真实信号无关的尖锐不连续，从而引入广谱伪能量。于是代码先用 `PreImpute.fill` 生成 $\boldsymbol{X}_{\text{seed}}$：

- 对每个变量分别做一次正向填充和一次反向填充；
- 若某位置两边都存在观测值，则取前后填充值均值；
- 若只在一侧有观测值，则取单侧填充值；
- 两侧都没有历史可用值时，仍为零。

因此，$\boldsymbol{X}_{\text{seed}}$ 的作用是给频域专家提供一个更平滑、更适合做谱分析的起点，而不是让频域模块从大量零块中“直接猜测”频谱。

5 频域插补模块的数学原理

5.1 模块结构概述

`FrequencyOnlyReconstructor` 由两部分组成：

1. `FrequencyImputer`：先到频域，再做频谱增强，再回到时域；
2. `output_head`：对时域输出做一层跨变量残差修正。

对输入 $\boldsymbol{X}_{\text{seed}} \in \mathbb{R}^{B\times T\times N}$，其输出为

$$\boldsymbol{X}_{\text{freq}} \in \mathbb{R}^{B\times T\times N}. \tag{19}$$

5.2 实数 FFT 与半谱表示

模块先交换时间轴和变量轴：

$$\boldsymbol{X}^{\pi} = \text{permute}(\boldsymbol{X}_{\text{seed}}) \in \mathbb{R}^{B\times N\times T}. \tag{20}$$

然后沿时间轴做实数快速傅里叶变换：

$$\boldsymbol{Z} = \mathcal{F}_r(\boldsymbol{X}^{\pi}) \in \mathbb{C}^{B\times N\times K}, \tag{21}$$

其中

$$K = \left\lfloor \frac{T}{2} \right\rfloor + 1. \tag{22}$$

对当前默认长度 $T = 50$，有 $K = 26$。之所以只保留 26 个频点，是因为实值信号的完整离散频谱满足共轭对称性，正频率半谱已经足以唯一恢复原始时域信号。代码因此使用 `torch.fft.rfft` 而不是完整 FFT。

5.3 实部、虚部与频谱描述向量

把 $\boldsymbol{Z}$ 写成实部和虚部：

$$\boldsymbol{Z} = \boldsymbol{R} + j\boldsymbol{I}, \qquad \boldsymbol{R}, \boldsymbol{I} \in \mathbb{R}^{B\times N\times K}. \tag{23}$$

再按最后一维拼接，得到频谱描述向量：

$$\boldsymbol{H} = [\boldsymbol{R}, \boldsymbol{I}] \in \mathbb{R}^{B\times N\times 2K}. \tag{24}$$

这一表示的含义是：对每个变量，模块不直接建模幅值和相位，而是对复频谱的笛卡尔坐标 (R_k, I_k) 做学习。这样做有两个实际好处：

- 避免直接回归相位时常见的角度环绕问题；
- 对实部和虚部做残差修正，等价于在复平面上对每个频点做向量平移，因此会**同时改变幅值和相位**。

5.4 注意力分支：决定每个频点该修多少

注意力分支是一个两层感知机：

$$\boldsymbol{A} = \sigma(\boldsymbol{W}_{a2}\,\phi(\boldsymbol{W}_{a1}\boldsymbol{H} + \boldsymbol{b}_{a1}) + \boldsymbol{b}_{a2}) \in (0,1)^{B\times N\times K}, \tag{25}$$

其中 $\phi(\cdot)$ 是 ReLU，$\sigma(\cdot)$ 是 Sigmoid。

这里的 $\boldsymbol{A}$ 可以理解为**频点级门控**：如果某个频点的权重接近 0，则增强分支对该频点的修正几乎不会生效；如果接近 1，则增强分支给出的残差会完整加入原始频谱。

由于 MLP 是对最后一维做线性映射，因此它对每个变量分别处理频谱向量，但**参数在变量之间共享**。换句话说，模型假设“如何根据频谱形状做修正”的规则在不同变量之间具有共性。

5.5 增强分支：学习复数频谱残差

增强分支同样是一个两层感知机：

$$\boldsymbol{E} = \boldsymbol{W}_{e2}\,\phi(\boldsymbol{W}_{e1}\boldsymbol{H} + \boldsymbol{b}_{e1}) + \boldsymbol{b}_{e2} \in \mathbb{R}^{B\times N\times 2K}. \tag{26}$$

把它拆成两部分：

$$\boldsymbol{E} = [\boldsymbol{E}_R, \boldsymbol{E}_I], \qquad \boldsymbol{E}_R, \boldsymbol{E}_I \in \mathbb{R}^{B\times N\times K}. \tag{27}$$

再由注意力分支调制：

$$\Delta\boldsymbol{Z} = (\boldsymbol{E}_R \odot \boldsymbol{A}) + j(\boldsymbol{E}_I \odot \boldsymbol{A}). \tag{28}$$

最终增强后的频谱为

$$\widetilde{\boldsymbol{Z}} = \boldsymbol{Z} + \Delta\boldsymbol{Z}. \tag{29}$$

这个表达式体现出该模块是**残差式频谱校正**，而不是重新从零生成频谱。残差式的好处是：

- 原始种子频谱 $\boldsymbol{Z}$ 作为锚点保留下来，训练更稳定；
- 模块只需学习“应该偏离多少”，而不必重建完整谱形；
- 当输入已经足够可信时，注意力可把修正幅度压小。

5.6 逆变换回时域

增强后的半谱通过实数逆 FFT 还原到时域：

$$\widetilde{\boldsymbol{X}}^{\pi} = \mathcal{F}_r^{-1}(\widetilde{\boldsymbol{Z}}) \in \mathbb{R}^{B\times N\times T}. \tag{30}$$

代码中使用 `torch.fft.irfft(...,n=x.size(1))`，显式指定输出长度为原窗口长度 T，从而保证恢复后序列长度严格与输入窗口一致。

最后再交换轴得到

$$\widetilde{\boldsymbol{X}} \in \mathbb{R}^{B\times T\times N}. \tag{31}$$

5.7 输出头：把频域重构再做一次跨变量修正

`FrequencyOnlyReconstructor` 并没有直接返回 $\widetilde{\boldsymbol{X}}$，而是做了：

$$\boldsymbol{X}_{\text{freq}} = \widetilde{\boldsymbol{X}} + h_{\text{out}}(\widetilde{\boldsymbol{X}}), \tag{32}$$

其中 h_{out} 是

$$\mathbb{R}^N \to \mathbb{R}^N \tag{33}$$

的两层 MLP，在每个时间步上独立作用于 N 维变量向量。

这意味着：

- `FrequencyImputer` 负责**沿时间轴**的频谱建模；
- `output_head` 负责**沿变量轴**的跨变量耦合修正；
- 因而频域专家不是纯粹“每个变量各做各的”，其最后输出仍会利用变量间相关性。

5.8 默认配置下的具体维度

当前默认配置 $T = 50$、$N = 18$。于是频域模块内部最重要的中间量维度如下：

张量	一般形状	当前默认形状
$\boldsymbol{X}_{\text{seed}}$	$B \times T \times N$	$B \times 50 \times 18$
$\boldsymbol{X}^{\pi}$	$B \times N \times T$	$B \times 18 \times 50$
$\boldsymbol{Z}$	$B \times N \times K$ 复数	$B \times 18 \times 26$ 复数
$\boldsymbol{H}$	$B \times N \times 2K$	$B \times 18 \times 52$
$\boldsymbol{A}$	$B \times N \times K$	$B \times 18 \times 26$
$\boldsymbol{E}$	$B \times N \times 2K$	$B \times 18 \times 52$
$\widetilde{\boldsymbol{X}}$	$B \times T \times N$	$B \times 50 \times 18$
$\boldsymbol{X}_{\text{freq}}$	$B \times T \times N$	$B \times 50 \times 18$

在当前默认配置下，`FrequencyOnlyReconstructor` 的可训练参数总数为 24314。

6 从代码看张量如何流动

6.1 训练时的逐步张量流

训练循环的一个 batch 中，主要张量按如下顺序流动：

1. $\boldsymbol{X}_{\text{true}}, \boldsymbol{M}_s \in \mathbb{R}^{B\times 50\times 18}$ 从 DataLoader 读出；
2. 采样得到 $\boldsymbol{M}_h \in \mathbb{R}^{B\times 50\times 18}$；
3. $\boldsymbol{M} = \max(\boldsymbol{M}_s, \boldsymbol{M}_h)$；
4. $\boldsymbol{X}_{\text{input}} = \boldsymbol{X}_{\text{true}} \odot (1 - \boldsymbol{M})$；
5. $\boldsymbol{X}_{\text{seed}} = \text{PreImpute}(\boldsymbol{X}_{\text{input}}, \boldsymbol{M})$；
6. GCN 专家输出 $\boldsymbol{X}_{\text{gcn}} \in \mathbb{R}^{B\times 50\times 18}$，并给出邻接矩阵 $\boldsymbol{A}_{\text{graph}} \in \mathbb{R}^{B\times 18\times 18}$；

7. 频域专家输出 $\boldsymbol{X}_{\text{freq}} \in \mathbb{R}^{B\times 50\times 18}$；
8. 采样率嵌入扩展为 $\boldsymbol{E}_{\text{rate}} \in \mathbb{R}^{B\times 50\times 18\times 8}$；
9. 门控特征堆叠为

$$[\boldsymbol{X}_{\text{seed}}, \boldsymbol{M}, \boldsymbol{X}_{\text{gcn}}, \boldsymbol{X}_{\text{freq}}, \boldsymbol{X}_{\text{gcn}} - \boldsymbol{X}_{\text{seed}}, \boldsymbol{X}_{\text{freq}} - \boldsymbol{X}_{\text{seed}}] \in \mathbb{R}^{B\times 50\times 18\times 6}; \tag{34}$$

10. 与采样率嵌入拼接后，门控输入为 $\mathbb{R}^{B\times 50\times 18\times 14}$；
11. 门控 logits 为 $\mathbb{R}^{B\times 50\times 18\times 2}$，softmax 后得到两专家权重；
12. 专家堆叠 $[\boldsymbol{X}_{\text{gcn}}, \boldsymbol{X}_{\text{freq}}]$ 后，按最后一维加权求和，得到 $\boldsymbol{X}_{\text{imp}} \in \mathbb{R}^{B\times 50\times 18}$；
13. 把模型缺失位置替换为插补值，得到完整窗口 $\boldsymbol{X}_{\text{complete}}$；
14. $\boldsymbol{X}_{\text{complete}}$ 再送入检测器，输出 $\boldsymbol{X}_{\text{det}} \in \mathbb{R}^{B\times 50\times 18}$。

6.2 门控层为什么会用到频域残差

这一点很值得解释。门控输入不是只看两个专家的绝对输出，还显式使用了

$$\boldsymbol{X}_{\text{gcn}} - \boldsymbol{X}_{\text{seed}}, \qquad \boldsymbol{X}_{\text{freq}} - \boldsymbol{X}_{\text{seed}}. \tag{35}$$

这等价于让门控网络看到“每个专家相对种子值改动了多少”。对频域专家来说，这个设计的意义是：

- 若频域修正很小，说明种子本身已经可信，门控可以更保守；
- 若频域修正很大，门控会显式感知到该专家在该位置做了较强重构；
- 这样门控决策不只依赖输出值本身，还依赖“改动幅度”这一置信度线索。

6.3 测试时的张量流有何不同

测试时不再采样 holdout 掩码，直接令

$$\boldsymbol{M} = \boldsymbol{M}_s. \tag{36}$$

也就是说：

- 训练时有两种缺失：结构性缺失和人为 holdout；
- 测试时只有结构性缺失，不再额外遮挡观测值。

但频域专家的前向结构**完全相同**：仍然是 $\boldsymbol{X}_{\text{input}} \to \boldsymbol{X}_{\text{seed}} \to \boldsymbol{X}_{\text{freq}}$。

7 训练入口、损失函数与参数更新

7.1 独立训练入口：独立频域脚本

独立脚本中的训练逻辑非常直接。对每个 batch：

1. 从原始缺失掩码 $\boldsymbol{M}_s$ 中找到观测位置；
2. 在观测位置上随机采样 `random_drop`，比例由 `mask_drop_rate` 控制；
3. 把 $\boldsymbol{M}_s$ 与 `random_drop` 合并为输入掩码；
4. 把这些位置清零后送入频域模块；
5. 只在 `random_drop` 对应的位置上计算重构 MSE。

若某个 batch 恰好没有采到任何 `random_drop` 位置，代码就退化为在全部观测位置上计算损失。这个逻辑保证了每个 batch 都有监督信号。

对应目标函数可写为

$$\mathcal{L}_{\text{standalone}} = \frac{\|(\boldsymbol{X}_{\text{freq}} - \boldsymbol{X}_{\text{true}}) \odot \boldsymbol{M}_{\text{target}}\|_F^2}{\max(\sum \boldsymbol{M}_{\text{target}}, 1)}. \tag{37}$$

7.2 联合训练入口：mra.py

`mra.py` 的训练入口更重要，因为这才是完整系统真正运行的路径。它与独立脚本的差别有三点：

- 频域模块不再单独前向，而是作为门控双专家的一部分；
- 优化目标不只包含频域重构损失，还包含融合损失、GCN 损失、图正则和检测器损失；
- 优化器对**整个模型的全部参数**统一做 AdamW 更新。

训练期一个 batch 的伪代码可以概括为：

1. 读入 $(\boldsymbol{X}_{\text{true}}, \boldsymbol{M}_s)$；
2. 采样 holdout 掩码 $\boldsymbol{M}_h$；
3. 令 $\boldsymbol{M} = \max(\boldsymbol{M}_s, \boldsymbol{M}_h)$，构造 $\boldsymbol{X}_{\text{input}}$；
4. 前向得到 $\boldsymbol{X}_{\text{gcn}}, \boldsymbol{X}_{\text{freq}}, \boldsymbol{X}_{\text{imp}}, \boldsymbol{X}_{\text{complete}}, \boldsymbol{A}_{\text{graph}}, \boldsymbol{X}_{\text{det}}$；
5. 计算五项损失并线性加权求和；
6. `zero_grad`；
7. `total_loss.backward()`；
8. 全模型梯度裁剪到 1.0；
9. `optimizer.step()`。

7.3 五项损失的精确定义

1. 融合损失

$$\mathcal{L}_{\text{fusion}} = \frac{\|(\boldsymbol{X}_{\text{imp}} - \boldsymbol{X}_{\text{true}}) \odot \boldsymbol{M}_h\|_F^2}{\max(\sum \boldsymbol{M}_h, 1)}. \tag{38}$$

它只在额外 holdout 的观测位置上监督最终门控融合后的插补结果。

2. GCN 专家损失

$$\mathcal{L}_{\text{gcn}} = \frac{\|(\boldsymbol{X}_{\text{gcn}} - \boldsymbol{X}_{\text{true}}) \odot \boldsymbol{M}_h\|_F^2}{\max(\sum \boldsymbol{M}_h, 1)}. \tag{39}$$

3. 频域专家损失

$$\mathcal{L}_{\text{freq}} = \frac{\|(\boldsymbol{X}_{\text{freq}} - \boldsymbol{X}_{\text{true}}) \odot \boldsymbol{M}_h\|_F^2}{\max(\sum \boldsymbol{M}_h, 1)}. \tag{40}$$

这项损失的直接作用是让频域专家在被人为挖掉的观测位置上学会恢复真实值。由于真实结构性缺失没有标签，所以代码**只能通过 holdout 自监督**来给频域模块提供可验证目标。

4. 图正则损失

$$\mathcal{L}_{\text{graph}} = \frac{1}{BN} \sum_{b=1}^{B} \sum_{n=1}^{N} \left(A_{b,nn}^{\text{graph}} - \tau \right)^2, \tag{41}$$

其中 τ 对应 `diag_target`。

5. 检测器损失 令

$$\boldsymbol{O} = 1 - \boldsymbol{M}_s \tag{42}$$

为结构性观测掩码，代码中使用

$$\boldsymbol{X}_{\text{target}} = \text{sg}(\boldsymbol{X}_{\text{complete}}) \tag{43}$$

作为停止梯度的目标，于是

$$\mathcal{L}_{\text{det}} = \frac{\|(\boldsymbol{X}_{\text{det}} - \text{sg}(\boldsymbol{X}_{\text{complete}})) \odot \boldsymbol{O}\|_F^2}{\max(\sum \boldsymbol{O}, 1)}. \tag{44}$$

总损失 默认权重为

$$(\lambda_f, \lambda_g, \lambda_r, \lambda_d, \lambda_a) = (1.0, 0.4, 0.4, 0.5, 0.1), \tag{45}$$

因此

$$\mathcal{L}_{\text{total}} = \lambda_f \mathcal{L}_{\text{fusion}} + \lambda_g \mathcal{L}_{\text{gcn}} + \lambda_r \mathcal{L}_{\text{freq}} + \lambda_d \mathcal{L}_{\text{det}} + \lambda_a \mathcal{L}_{\text{graph}}. \tag{46}$$

7.4 哪些参数会被哪些损失更新

这是用户最关心、也最容易读错代码的地方。先定义几组参数：

- θ_{freq}：频域专家参数，包括 `attention`、`freq_enhance` 和 `output_head`；
- θ_{gate}：门控 MLP 与采样率嵌入参数；
- θ_{gcn}：GCN 专家参数与图学习参数；
- θ_{det}：Transformer 检测器参数。

结合代码依赖关系与一次单 batch 自动求导检查，可得到下表：

损失项	更新 θ_{freq}	更新 θ_{gate}	更新 θ_{gcn}	更新 θ_{det}
$\mathcal{L}_{\text{fusion}}$	是	是	是	否
$\mathcal{L}_{\text{freq}}$	是	否	否	否
$\mathcal{L}_{\text{gcn}}$	否	否	是	否
$\mathcal{L}_{\text{det}}$	是	是	是	是
$\mathcal{L}_{\text{graph}}$	否	否	是	否

7.5 为什么检测器损失会更新频域模块

这点必须单独展开。很多人看到

$$\boldsymbol{X}_{\text{target}} = \text{sg}(\boldsymbol{X}_{\text{complete}}) \tag{47}$$

会误以为检测器损失不会影响插补器。其实不是。

检测器输出满足

$$\boldsymbol{X}_{\text{det}} = D(\boldsymbol{X}_{\text{complete}}) + \boldsymbol{X}_{\text{complete}}, \tag{48}$$

其中 $D(\cdot)$ 表示 Transformer 编码和重构头产生的残差项。于是

$$\mathcal{L}_{\text{det}} = \|(D(\boldsymbol{X}_{\text{complete}}) + \boldsymbol{X}_{\text{complete}} - \text{sg}(\boldsymbol{X}_{\text{complete}})) \odot \boldsymbol{O}\|_F^2 . \tag{49}$$

虽然数值上 $\boldsymbol{X}_{\text{complete}} - \text{sg}(\boldsymbol{X}_{\text{complete}}) = \boldsymbol{0}$，但梯度上并不是零，因为 $\text{sg}(\cdot)$ 切断的是目标分支，不是输入分支。对 $\boldsymbol{X}_{\text{complete}}$ 求导时仍然有：

$$\frac{\partial \mathcal{L}_{\text{det}}}{\partial \boldsymbol{X}_{\text{complete}}} \neq \boldsymbol{0}. \tag{50}$$

因此，$\mathcal{L}_{\text{det}}$ 会沿着

$$\boldsymbol{X}_{\text{complete}} \to \boldsymbol{X}_{\text{imp}} \to \boldsymbol{X}_{\text{freq}} \tag{51}$$

这条路径反向传播到频域专家和门控网络。实际单 batch 自动求导检查也验证了这一点：`detector_loss` 对频域专家、门控层、GCN 专家和检测器四组参数都产生了非零梯度。

7.6 训练时额外遮挡的意义

`sample_holdout_mask` 只在观测位置上采样额外遮挡，并保证**每个样本至少有一个 holdout 位置**。它的理论作用是把原本没有标签的插补问题转化为可监督的自编码任务：

- 结构性缺失位置没有真实标签，无法直接监督；
- 额外挖掉的观测位置原本有真实值，因而可以拿来构造监督信号；
- 频域模块于是学习的是“对真实缺失模式有代表性的随机观测缺失恢复能力”。

8 训练集学到的参数，在测试集里如何起作用

8.1 先说参数本身

在训练集上被更新并保存到 `model.pt` 的，与频域模块直接相关的参数主要有：

- 频点注意力分支参数；
- 频谱增强分支参数；
- 输出头参数；
- 与频域模块协同工作的门控参数和采样率嵌入参数；
- 影响频域模块上下文作用方式的检测器参数。

测试阶段这些参数不会再更新，而是固定用于前向推理。

8.2 测试阶段的直接作用

训练得到的频域参数在测试集上有三条直接作用路径：

1. 直接生成频域专家输出 对每个测试窗口，固定参数的频域专家会把 $\boldsymbol{X}_{\text{seed}}$ 映射成 $\boldsymbol{X}_{\text{freq}}$。这是频域模块最直接的输出。

2. 通过门控参与最终补全 测试时门控层会读取 $\boldsymbol{X}_{\text{freq}}$、$\boldsymbol{X}_{\text{gcn}}$、采样率嵌入和差值特征，决定每个时间步每个变量该更信任哪位专家。因此频域参数不仅决定自己输出什么，还决定门控更愿不愿意信它。

3. 作为检测器输入的一部分 $\boldsymbol{X}_{\text{complete}}$ 送入 Transformer 检测器，检测器对观测位置的重构误差构成 `detector_score`。因此频域模块会通过完整序列上下文影响检测器对窗口的理解。

8.3 测试阶段的间接作用

训练得到的频域参数还通过统计量构造对最终判决产生间接影响：

1. 影响专家分歧分数 代码定义

$$\text{disagreement_score} = \frac{1}{TN}\sum_{t=1}^{T}\sum_{n=1}^{N}\left|X_{\text{gcn},t,n} - X_{\text{freq},t,n}\right|. \tag{52}$$

只要频域专家的输出改变，这个分数就会改变。

2. 影响门控熵 门控熵定义为

$$\text{gate_entropy} = -\frac{1}{TN}\sum_{t=1}^{T}\sum_{n=1}^{N}\sum_{e=1}^{2}\Pi_{t,n,e}\log\Pi_{t,n,e}. \tag{53}$$

而 $\boldsymbol{\Pi}$ 的生成又依赖 $\boldsymbol{X}_{\text{freq}}$，因此频域参数会改变门控不确定性。

3. 影响分布偏移分数 `compute_distribution_shift_scores` 不是直接对原始测试窗口打分，而是对**补全后的训练序列和测试序列**再次窗口化后抽取统计特征，再计算 z-score 幅度：

$$\text{shift_score} = \sqrt{\frac{1}{D_f}\sum_{d=1}^{D_f} z_d^2}. \tag{54}$$

这些特征里甚至还包含了一个简化的窗口 FFT 幅值统计，因此频域模块会双重影响 `shift_score`：

- 先改变补全后的时域序列；
- 再改变从该时域序列计算出来的频域统计特征。

4. 影响训练统计基准与阈值 最终分数归一化和阈值选择依赖**训练集统计量**：

- `normalize_scores` 用训练集均值与标准差归一化训练/测试分数；
- `choose_threshold` 用训练集平滑后分数的高斯上界与分位数共同确定阈值。

由于这些训练分数本身来自训练好后的频域模块，因此训练集中学到的频域参数不仅影响测试样本的分子，也影响用来判定异常的统计基准。

8.4 一个经常被忽略的事实

测试输出的 `train_completed.csv` 和 `test_completed.csv` 并不是“窗口内所有时间步的平均结果”，而是对每个窗口只取最后一个时间步：

$$\boldsymbol{x}_k^{\text{out}} = \boldsymbol{X}_{\text{complete}}[k, -1, :]. \tag{55}$$

之所以这样做，是因为 `build_windows` 对每个原始时间点都构造了一个以该点为结尾的窗口。于是窗口最后一步自然与原始时间轴一一对应。频域模块因此对导出补全序列的影响，最终体现在每个时间点对应窗口末端的输出上。

9 默认配置下的参数、张量和评分链条

9.1 与频域模块直接相关的默认超参数

参数名	默认值	作用
`seq_len`	50	窗口长度，决定频谱分辨率与 `freq_len=26`
`epochs`	12	联合训练轮数
`batch_size`	64	batch 大小
`lr`	10^{-3}	AdamW 学习率
`weight_decay`	10^{-4}	AdamW 权重衰减
`holdout_ratio`	0.15	训练期额外遮挡比例
`gate_hidden_dim`	64	门控 MLP 隐层维度
`rate_embed_dim`	8	采样率嵌入维度
`score_disagreement_weight`	0.25	专家分歧分数权重
`score_shift_weight`	1.0	分布偏移分数权重

9.2 最终异常分数是怎样把频域模块卷进去的

最终原始分数满足

$$s = |z_{\text{det}}| + \alpha z_{\text{gap}} + \beta z_{\text{shift}}, \tag{56}$$

其中 α 对应 `score_disagreement_weight`，β 对应 `score_shift_weight`。

频域模块分别通过以下方式进入这三个分量：

- $|z_{\text{det}}|$：经由 $\boldsymbol{X}_{\text{complete}}$ 改变检测器输入；
- z_{gap}：直接由 $\boldsymbol{X}_{\text{freq}}$ 与 $\boldsymbol{X}_{\text{gcn}}$ 的差决定；
- z_{shift}：经由补全后的训练/测试序列改变统计特征和其 z-score。

所以频域模块不是“只负责补全，不参与检测”；它实际上深度嵌入了最终异常分数的三条构成路径。

10 设计思路、优点与局限

10.1 设计思路总结

从代码可以看出，作者对频域模块的设计思路大致是：

1. 先用时域双向填充消除硬零掩码造成的频谱伪影；
2. 再把时间维信号映射到半谱域，用共享参数学习“应该修哪些频点、修多少”；
3. 采用残差式复谱修正，保证训练稳定；
4. 回到时域后再做一次跨变量 MLP 修正，补上频谱分支中较弱的变量相关性建模；
5. 不在频域专家内部直接塞采样率信息，而是把采样率信息留给门控层和检测器使用，从而把“频谱修正规则”设计成更通用、更共享的形式。

10.2 这样设计的优点

- **参数高效**：核心频谱 MLP 在变量间共享，不需要每个变量一套频域网络。
- **兼顾稳定性**：残差式频谱增强比直接生成完整频谱更稳。
- **适合多采样率场景**：频域专家专注于从时序模式里恢复慢变量，高频/低频之间的差异由门控和检测器的 rate embedding 进一步调节。
- **容易与其他专家融合**：频域输出与图输出保持同形状，便于逐位置门控融合。

10.3 从代码出发能看到的局限

- **频域专家本身不显式使用掩码**：它只看 $\boldsymbol{X}_{\text{seed}}$，掩码信息由门控层而不是频谱 MLP 直接消费，因此频域分支对“当前位置到底原生缺失还是人为 holdout”并不完全显式敏感。
- **频率分辨率受窗口长度限制**：$T = 50$ 时只有 26 个半谱频点，适合粗粒度频谱修正，不适合非常细的谱线辨识。
- **变量特异性较弱**：频域核心参数在所有变量间共享，若不同变量的动态机理差异很大，单一共享频谱修正规则可能不够细。
- **检测器损失会反推插补器**：这有利于联合最优，但也意味着频域模块学到的并不一定是“纯插补意义下最优”，而是“更有利于后续检测器的补全表示”。
- **输出头依赖固定变量顺序**：`output_head` 在最后一维做全连接，默认假设 18 维变量的排列顺序固定且具有稳定含义。

11 结论

回到本文最初的问题，可以把答案压缩成一句话：

> `mra.py` 中的频域插补模块本质上是一个“以双向时域种子为输入、以实数 FFT 半谱为中介、以残差复谱增强为核心、再通过门控与 GCN 专家融合、并与异常检测器联合训练”的多采样率插补专家。

它的训练入口在 `mra.py:548-621`，但模块本身定义在 `freq_reconstruction.py:95-141`；它在训练时直接受 `freq_loss` 监督，也通过 `fusion_loss` 和 `detector_loss` 接受间接监督；它在测试时不仅产生补全值，还通过专家分歧、门控行为、分布偏移和训练统计基准共同影响最终异常分数。

如果后续还要继续扩展这份手册，最值得补写的方向有两个：

- 对频域专家与 GCN 专家在不同采样率变量上的门控偏好做统计实验；
- 对 `detector_loss` 是否应该反向更新插补器做消融实验，以区分“更利于补全”与“更利于检测”这两种训练目标。